Mobilemcp

Driving the mSpace Android app from an AI client, with every request and log in one place

Team guide: how it works, how to set it up, and what each of us needs to resolve on our own machine

Item	Detail
Project	Mobilemcp (fork of mobile-next/mobile-mcp with our logging and dev-viewer additions)
Get the code	git clone -b feature/dev-logs-viewer https://github.com/<your-org>/mobile-mcp.git Mobilemcp
Target app	mSpace Android app (package com.mli.mspace), Axis Max Life UAT backend
Audience	QA automation and mobile engineers who will run or extend the automation
Version	1.0, 4 November 2026
Status	Working on one Windows 11 machine with an Android emulator; ready for the team to replicate

> *In one line: an AI client sends tool calls to a small Node server, the server drives the Android emulator through adb, and a single terminal shows what was asked, what the device did and what the app called on the backend.*

1. What this is and why we built it

Testing mSpace by hand is slow and hard to reproduce. mobile-mcp is an open-source Model Context Protocol (MCP) server that lets an AI client such as Claude Code or Cursor operate a phone: launch apps, tap, type, swipe, read the screen and take screenshots. We cloned it as the Mobilemcp project, built it, and ran it against the mSpace app on an Android emulator.

Out of the box the server is silent about what it does, and the app's own API traffic lives in a separate logcat stream. We added two things so a tester can follow a run without juggling windows:

- **Request and response logging** on the server's HTTP endpoint, so every tool call and its reply is printed in full.
- **A single-terminal log viewer** that merges those server logs with the app's logcat output, parses the app's API request and response blocks, filters emulator noise, clears itself when the app is rerun, and lets you search everything with a keypress.

This guide explains the moving parts, the setup steps, and the dependencies each team member has to sort out locally before it works for them.

2. How it works

2.1. Components and data flow

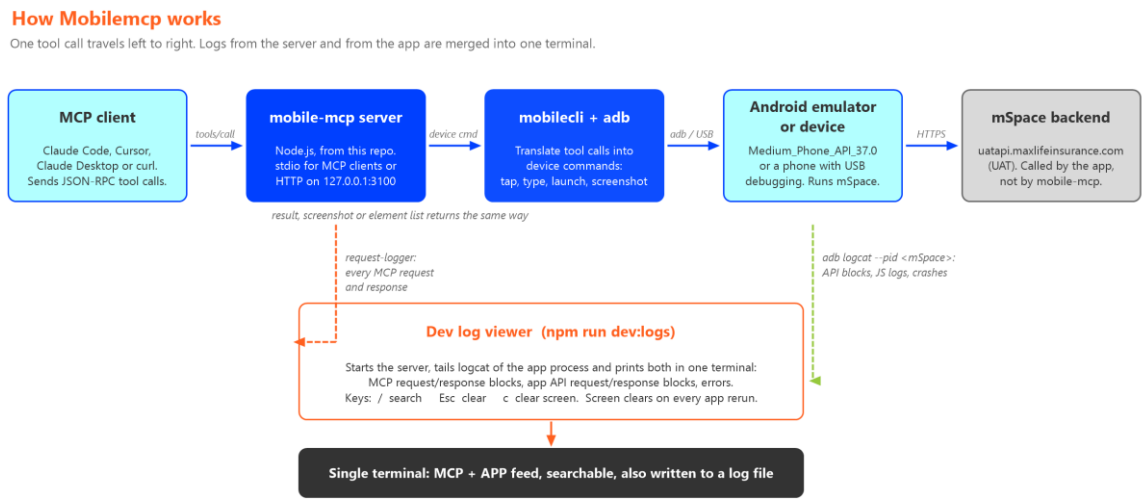


Figure 1. Components, the path of one tool call, and the two log feeds that the dev viewer merges.

Component	What it does	Where it runs
MCP client	Sends JSON-RPC tool calls such as mobile_launch_app or mobile_click_on_screen_at_coordinates. Claude Code, Cursor, Claude Desktop, or plain curl against the HTTP endpoint.	Your laptop
mobile-mcp server	Node.js process built from this repo (lib/index.js). Two modes: stdio (what MCP clients use) or Streamable HTTP on 127.0.0.1:3100/mcp (what curl and the dev viewer use).	Your laptop
mobilecli and adb	Bundled helper binary plus the Android Debug Bridge from the Android SDK. They turn a tool call into device commands and return screenshots or the UI element tree.	Your laptop, talking to the device
Android emulator or device	An Android Virtual Device (we use Medium_Phone_API_37.0) or a physical phone with USB debugging. mSpace must be installed on it.	Your laptop or USB
mSpace app and its backend	The React Native app calls uatapi.maxlifeinsurance.com itself. mobile-mcp never talks to that backend; we only observe those calls through logcat.	Device plus UAT network
Dev log viewer	scripts/dev-logs.mjs, run with npm run dev:logs. Starts the server, tails logcat for the mSpace process and prints both feeds in one terminal with search.	Your laptop

2.2. Life of one tool call

1. The client posts a tools/call request, for example mobile_click_on_screen_at_coordinates with a device id and x, y.
2. The request-logger middleware prints the request body with a sequence number.
3. The tool handler resolves the device by its id. For emulators the id is the AVD name (Medium_Phone_API_37.0), not the adb serial.
4. mobilecli or adb executes the action on the device: input tap, am start, screencap, uiautomator dump and so on.

5. The result text, screenshot or element list is returned as the JSON-RPC response, and the middleware prints it with the HTTP status and elapsed time.
6. Meanwhile the app reacts: it calls its backend, logs the API request and response to logcat, and the viewer prints that block next to the tool call that caused it.

> *Device id matters: a call made with the adb serial emulator-5554 is rejected with 'device not found'. Always use the id returned by mobile_list_available_devices.*

2.3. The two run modes

Mode	Command	Use it when
stdio	npm start (or: node lib/index.js)	An MCP client launches the server itself. Claude Code, Cursor and Claude Desktop do this. Nothing is printed to a terminal because stdin and stdout carry the protocol.
HTTP	node lib/index.js --listen 127.0.0.1:3100	You want to drive it with curl, a script, or watch it in the dev viewer. The dev viewer starts this mode for you.

3. What we added on top of mobile-mcp

3.1. Request and response logging

File: **src/request-logger.ts**, wired into **src/index.ts**. It is an Express middleware on the /mcp route. For every request it prints the JSON body; for every response it captures the Server-Sent Events stream, extracts the JSON-RPC payload and prints it with the status and the time taken. Set **MOBILEMCP_LOG_REQUESTS=0** to switch it off.

3.2. Single-terminal dev log viewer

File: **scripts/dev-logs.mjs**, run with **npm run dev:logs**. It replaces three separate windows (server, server log file, adb logcat) with one feed.

Feature	What you see
MCP blocks	One compact block per tool call: tool name, arguments, result, HTTP status and duration. Long results such as the element list are cut to 12 lines.

Feature	What you see
APP API blocks	The app's own API logs parsed into method, path, status, duration, screen name, request body and response body. Headers are hidden unless you pass --headers, and then tokens are redacted.
App log lines	React Native INFO, DEBUG and ERROR lines, Java exceptions, crashes. Emulator chatter such as garbage-collection stats and graphics warnings is filtered out. Stack traces are collapsed to three frames.
Follows reruns	When mSpace is killed and relaunched the viewer detects the new process, clears the screen and starts a fresh run. The log file copy keeps all runs.
Search	Press / and type a term: only matching entries are shown, whole blocks stay together, matches are highlighted, and earlier entries are included. Esc clears it.
Labelled errors	Anything a child process writes to stderr is shown with an ADB or ADB LOGCAT label, so a crash message can never appear without a source.

3.2.1. Keys while the viewer is running

Key	Action
/	Start a search. Type the term, press Enter.
Esc	Clear the search and show everything again. While typing, cancel.
c	Clear the screen. History is kept, so search still covers it.
?	Show help.
Ctrl+C	Quit. The server stops with it.

3.2.2. Useful flags

Flag	Effect
--package <id>	Follow a different app. Default com.mli.mspace.
--device <serial>	Use a different adb device. Default emulator-5554.
--port <n>	HTTP port for the server. Default 3100.
--grep <term>	Start with a search already applied.
--full	No truncation, pretty-printed JSON bodies, all frames.

Flag	Effect
--headers	Show request headers with Authorization and API keys redacted.
--all-logcat	Show every logcat line from the app process.
--no-clear	Keep logs from earlier runs on screen.
--no-server	Do not start the server; only tail the app logs.
--log-file <path>	Also write a plain-text copy of everything, never cleared.

4. Setup and run flow

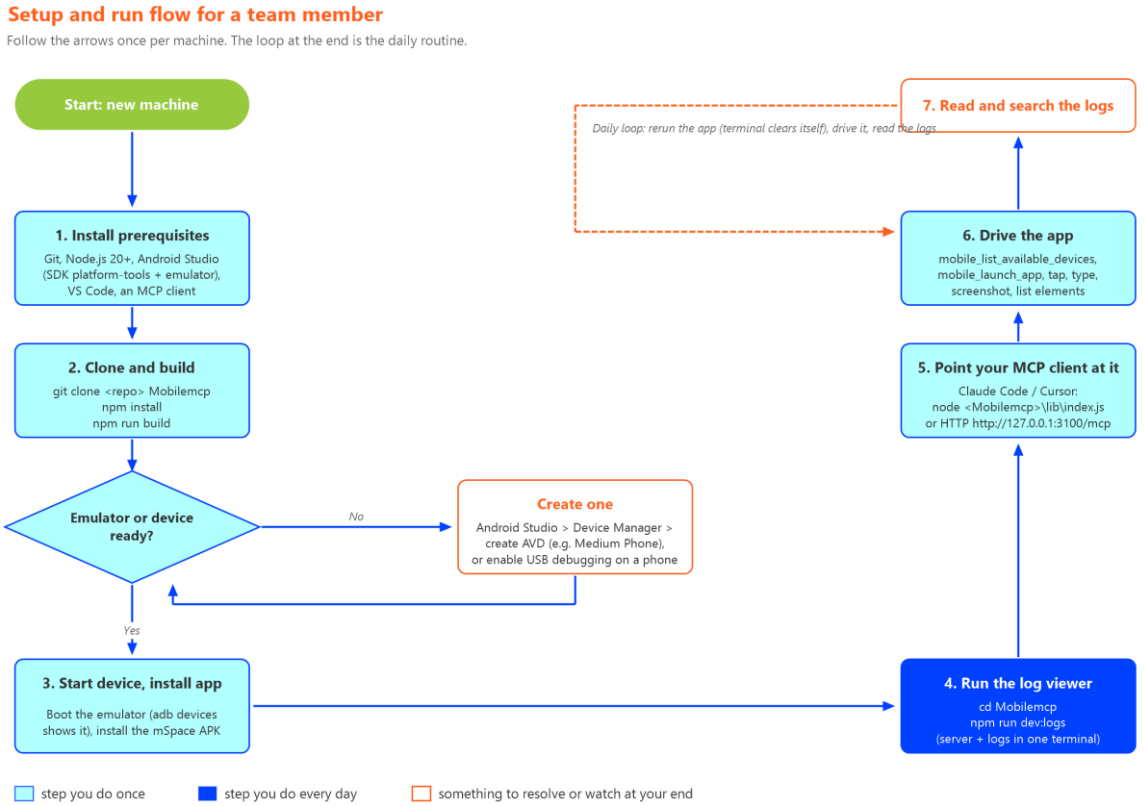


Figure 2. One-time setup on the left, daily routine on the right. Orange outlines mark the points where something must be resolved locally.

4.1. Step by step

4.1.1. One-time setup

1. Install the prerequisites in the table in the next section. Accept the Android SDK licences in Android Studio and make sure **adb** is on your PATH or at %LOCALAPPDATA%\Android\Sdk\platform-tools.
2. Clone the repository and build it:

```
git clone -b feature/dev-logs-viewer https://github.com/<your-org>/mobile-mcp.git Mobilemcp
cd Mobilemcp
npm install
npm run build
```

1. Create an Android Virtual Device in Android Studio (Device Manager, Medium Phone, a recent API level) or enable USB debugging on a phone. Confirm with **adb devices**.
2. Install the mSpace build on the device (adb install <apk>). Ask the mobile team for the current UAT APK.
3. Open the Mobilemcp folder in VS Code (or Visual Studio with the Node.js workload).

4.1.2. Every day

1. Start the emulator and wait until adb devices shows it as 'device', not 'offline'.
2. In the Mobilemcp folder run **npm run dev:logs**. The terminal shows the server start line and 'attached to com.mli.mspace' once the app is running.
3. Drive the app from your MCP client or with curl against http://127.0.0.1:3100/mcp. Start with mobile_list_available_devices and use the returned id in every call.
4. Read the feed. Press / to search, for example /login or /quotes, and Esc to go back.

4.2. Connecting an MCP client

Point your client at the local build instead of the published npx package so it picks up the logging changes. For Claude Code add a server entry like this (paths are examples):

```
"mobile-mcp": {
 "command": "node",
 "args": ["C:\\Users\\<you>\\Mobilemcp\\lib\\index.js"]
}
```

Cursor and Claude Desktop take the same command and args. If your client supports HTTP MCP servers, use http://127.0.0.1:3100/mcp while the dev viewer is running.

4.3. Smoke test

Check	Command or action	Expected result
Device visible	adb devices	emulator-5554 device
Server up	npm run dev:logs	'streamable http server listening on http://127.0.0.1:3100/mcp'
Device seen by server	tools/call mobile_list_available_devices	One device with id Medium_Phone_API_37.0 and state online
App launches	tools/call mobile_launch_app packageName=com.mli.mspace	'Launched app com.mli.mspace' and APP API blocks for cognito-token, device-id and check-version
Screen readable	tools/call mobile_list_elements_on_screen	Element list with refs, text, ids and coordinates
Search works	Press / then type cognito, Enter	Only the matching blocks are shown with the term highlighted

5. Dependencies you need to resolve at your end

Everything below is already working on the reference machine. Each team member has to put the same pieces in place locally. The third column tells you how to confirm it.

5.1. Software to install

Dependency	Why it is needed	How to check or get it
Git	Clone the repository and pull updates.	git --version. Install from git-scm.com.
Node.js 20 or newer, with npm	Runs the server and the dev viewer. The reference machine uses Node 24.	node --version. Install the LTS build from nodejs.org.
Android Studio with SDK platform-tools and emulator	Provides adb, the emulator binary, system images and the Device Manager to create an AVD.	Android Studio > SDK Manager: install 'Android SDK Platform-Tools' and 'Android Emulator'. Then adb --version.

Dependency	Why it is needed	How to check or get it
An Android Virtual Device, or a physical phone	The server needs something to drive. The emulator needs hardware acceleration (Hyper-V or Windows Hypervisor Platform) which may need admin rights to enable.	emulator -list-avds shows at least one AVD, or adb devices shows your phone with USB debugging on.
mSpace UAT build (APK)	The app under test. Not part of this repo.	Get the current APK from the mobile team and adb install it.
VS Code or Visual Studio	Editing and running the project. Visual Studio needs the Node.js development workload; VS Code works as is.	Open the Mobilemcp folder; the integrated terminal runs the npm scripts.
An MCP client	Claude Code, Cursor or Claude Desktop, if you want an AI to drive the app. curl is enough for manual tool calls.	Add the server entry shown in the previous section.
Windows Terminal or PowerShell	The dev viewer uses ANSI colours and raw keyboard input for search. Windows Terminal renders both well.	Run npm run dev:logs; the header line should be in colour and / should open the search prompt.

5.2. Access and approvals

Need	Who can resolve it	Notes
Repository access	Repo owner	Read access to https://github.com/<your-org>/mobile-mcp.git (branch feature/dev-logs-viewer). Ask the repo owner if the clone is refused. The original project lives at https://github.com/mobile-next/mobile-mcp.git, but that copy does not contain our changes.
mSpace UAT credentials	Mobile team / QA lead	The app logs in with an SSO ID. On the reference emulator 'Remember me' was on, so the app logged in by itself on every relaunch and hit the UAT login endpoint. Decide whether automated runs should do that.

Need	Who can resolve it	Notes
Network access to UAT	IT / VPN	The app must reach uatapi.maxlifeinsurance.com from the emulator. If you need VPN for UAT, the emulator uses the host's network, so connect the VPN first.
npm registry access	IT / proxy	npm install needs registry.npmjs.org. On a corporate proxy set HTTP_PROXY and HTTPS_PROXY for npm.
Admin rights for the emulator	IT	Enabling Hyper-V or Windows Hypervisor Platform and installing Android Studio may need local admin.
Optional: server authentication	Whoever exposes the HTTP port	The HTTP server accepts unauthenticated calls on localhost. If you ever bind it to a non-local address, set MOBILEMCP_AUTH to a bearer token and keep port 3100 behind the firewall.

5.3. Things we do not cover yet

- iOS. mobile-mcp supports iOS simulators and devices, but only from macOS with Xcode. Nothing in this guide was tested on iOS.
- Remote cloud devices. mobile-mcp can allocate real devices from its cloud fleet. We have not evaluated the cost or the data-handling implications, so leave those tools unused.
- Test scripts. Today the automation is driven interactively. Turning a flow into a repeatable script is the next step once the setup is on everyone's machine.

6. Known issues and troubleshooting

Symptom	Cause	What to do
'Device "emulator-5554" not found'	You passed the adb serial. The server identifies emulators by AVD name.	Use the id from mobile_list_available_devices, for example Medium_Phone_API_37.0.

Symptom	Cause	What to do
First call right after boot fails with 'device not found', later calls work	The device was still coming online.	Wait for adb devices to show 'device' (not 'offline') and retry.
Terminal shows '*** stack smashing detected ***: terminated' with no label	Seen twice on the reference machine. It did not come from any stream the viewer reads. Most likely a short-lived adb.exe crash.	Harmless so far. The viewer now labels every child's error output, so if it recurs the label tells you the source. Report it with the log file.
Port 3100 already in use; the server exits silently	An older server is still running.	Stop old node processes (Task Manager, or Get-Process node) or run the viewer with --port 3101.
check-version says the app version is no longer supported	The UAT backend expects a newer build than the one installed.	Install the current APK from the mobile team. This is an app or backend state, not a tooling fault.
The app logs in by itself on launch	'Remember me' is ticked with saved credentials.	Untick it in the app before the next restart if automated runs must not log in.
No APP lines in the viewer	The app process is not running, or the package name differs.	Launch the app; the viewer attaches within two seconds. Check with adb shell pidof com.mli.mspace.
Claude Code says the mobile-mcp server timed out at startup	The npx-based server entry downloads and starts slowly, or adb is not ready.	Point the client at the local build (node lib/index.js) as shown earlier.
Long warning lines from React Native or GFXSTREAM flood the feed	Emulator noise.	The viewer filters these by default. Use --all-logcat only when you really need everything.

7. Reference

7.1. Get the code

What	Where
Team repository (with all recent changes)	https://github.com/<your-org>/mobile-mcp.git
Branch to use	feature/dev-logs-viewer
Clone command	git clone -b feature/dev-logs-viewer https://github.com/<your-org>/mobile-mcp.git Mobilemcp
Update an existing clone	git fetch origin && git checkout feature/dev-logs-viewer && git pull
Original upstream project (no Monocept changes)	https://github.com/mobile-next/mobile-mcp.git

7.2. Files changed in this project

File	Purpose
src/request-logger.ts	Express middleware that logs every MCP request and response on the HTTP endpoint.
src/index.ts	Registers the middleware on the HTTP server.
scripts/dev-logs.mjs	Single-terminal viewer: server start, logcat follow, API block parsing, noise filter, clear on rerun, search.
package.json	Adds the dev:logs script.

7.3. Commands at a glance

```
npm install                    install dependencies
npm run build                  compile TypeScript to lib/
npm start                      server in stdio mode (for MCP clients)
node lib/index.js --listen 127.0.0.1:3100   server in HTTP mode
npm run dev:logs               server + app logs in one terminal
npm run dev:logs -- --grep login start with a search applied
npm run dev:logs -- --full --headers       everything, untruncated
```

7.4. Example tool call with curl

```
curl -X POST http://127.0.0.1:3100/mcp ^
  -H "Content-Type: application/json" ^
  -H "Accept: application/json, text/event-stream" ^
  -d "{\"jsonrpc\":\"2.0\",\"id\":1,\"method\":\"tools/call\",
      \"params\":{\"name\":\"mobile_launch_app\",

\"arguments\":{\"device\":\"Medium_Phone_API_37.0\",\"packageName\":\"com.mli.mspace
\"}}}"
```

7.5. Useful tools exposed by the server

Tool	What it does
mobile_list_available_devices	Lists emulators and devices with the id to use in every other call.
mobile_launch_app / mobile_terminate_app	Start or stop an app by package name.
mobile_list_elements_on_screen	Returns the UI tree with refs, text, ids and coordinates. Prefer this over screenshots for finding things to tap.
mobile_click_on_screen_at_coordinates / mobile_type_keys / mobile_swipe_on_screen	Interact with the screen.
mobile_take_screenshot / mobile_save_screenshot	Capture the screen, inline or to a file.
mobile_get_device_logs	Pull logcat with filters such as process=com.mli.mspace when you are not using the dev viewer.
mobile_batch_commands	Run a known sequence (tap, type, tap) in one round trip.

Questions or improvements: raise them on the feature/dev-logs-viewer branch so the fixes land in one place.